

Базовые сущности Kubernetes: ReplicaSet, Deployment, StatefulSet, DaemonSet

Microservice Architecture

REC **Проверить, идет ли запись**

Меня хорошо видно & слышно?





Майоров Антон

Старший разработчик ПО

- 10+ лет опыта разработки ПО на платформе .NET
- Специализация: приложения со сложной бизнес логикой
- Последние 4 года - старший системный разработчик в Ренессанс Страхование

@amaiorov

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе
#**канал группы**

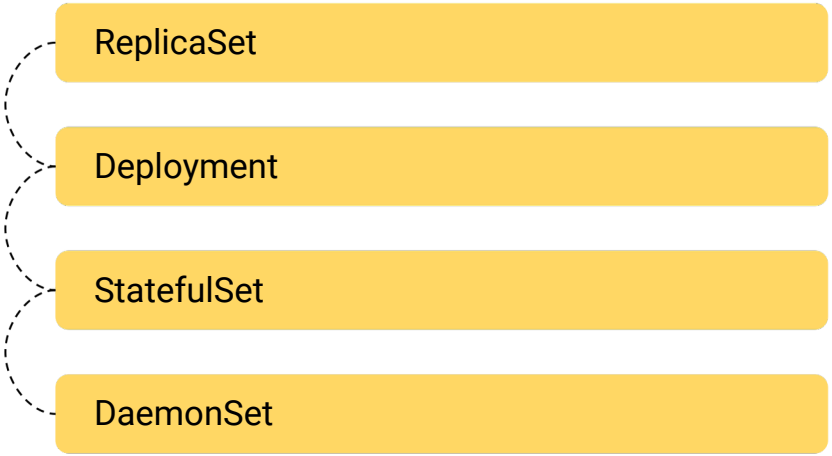


Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Маршрут вебинара



Цели вебинара

1. Познакомиться с сущностями ReplicaSet и Deployment
2. Создавать и использовать объекты k8s StatefulSet
3. Создавать и конфигурировать объект k8s DaemonSet

Контроллеры в k8s

Почему не деплоить просто pod?

Instance 1

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-kuber
spec:
  containers:
  - name: app
    image: roflmaoinmysoul/otus:v2.0
```

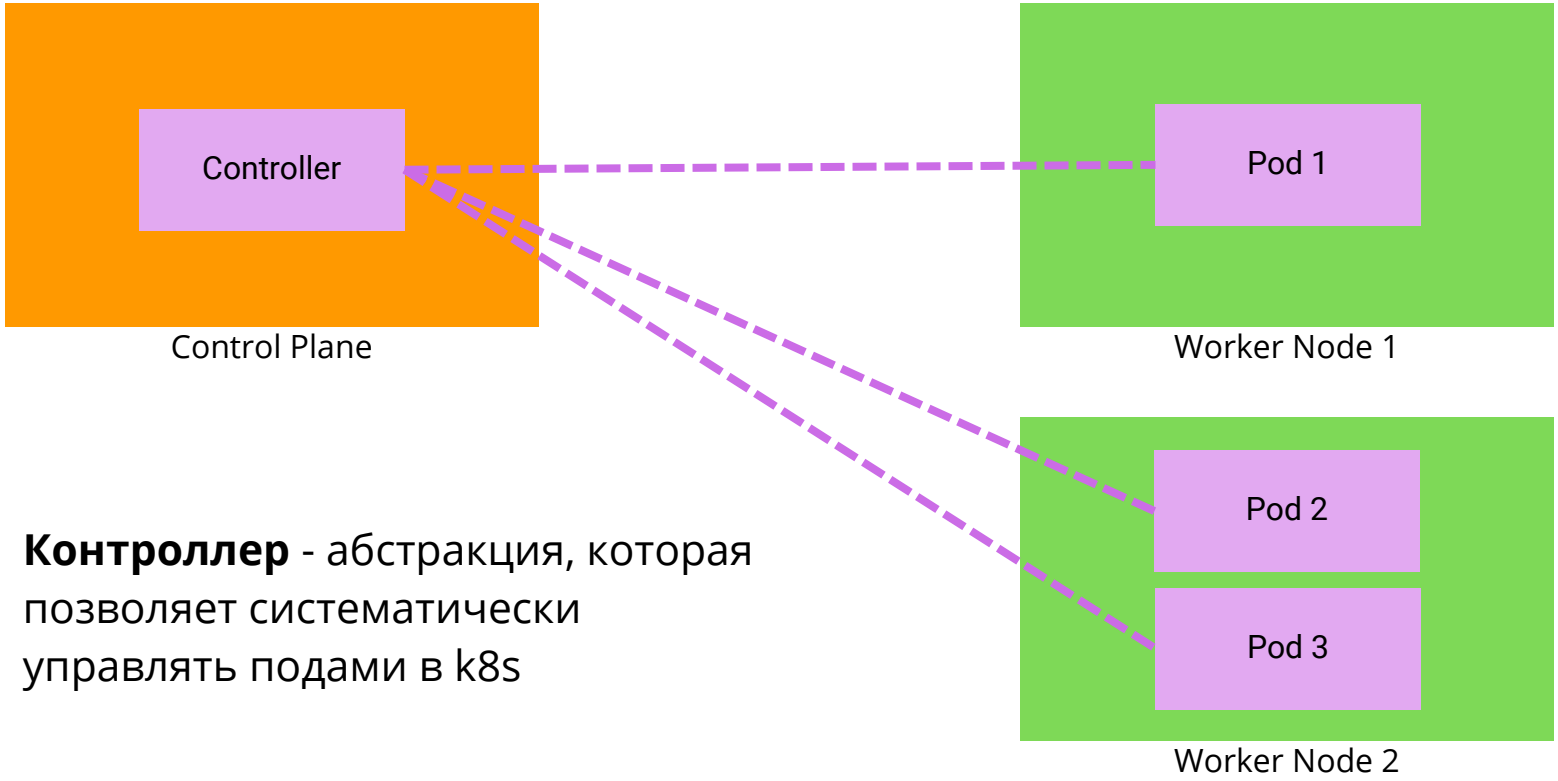
Instance 2

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-kuber-2
spec:
  containers:
  - name: app
    image: roflmaoinmysoul/otus:v2.0
```

Почему не деплоить просто pod?

- **отказоустойчивость** – за ЖЦ пода в общем случае следит kubelet, но есть много ситуаций когда под может быть удален и не будет перезапущен автоматически
- **уникальность имен** – каждый инстанс (под) должен иметь уникальное имя в рамках namespace
- **консистентность конфигурации всех инстансов** – необходимо самостоятельно менять конфигурацию всех подов одного приложения, обеспечивать чтобы у всех она была одинакова
- **управление обновлением** – выкатывать новую версию приложения тоже придется руками

Контроллеры



Контроллер - абстракция, которая позволяет систематически управлять подами в k8s

ReplicaSet

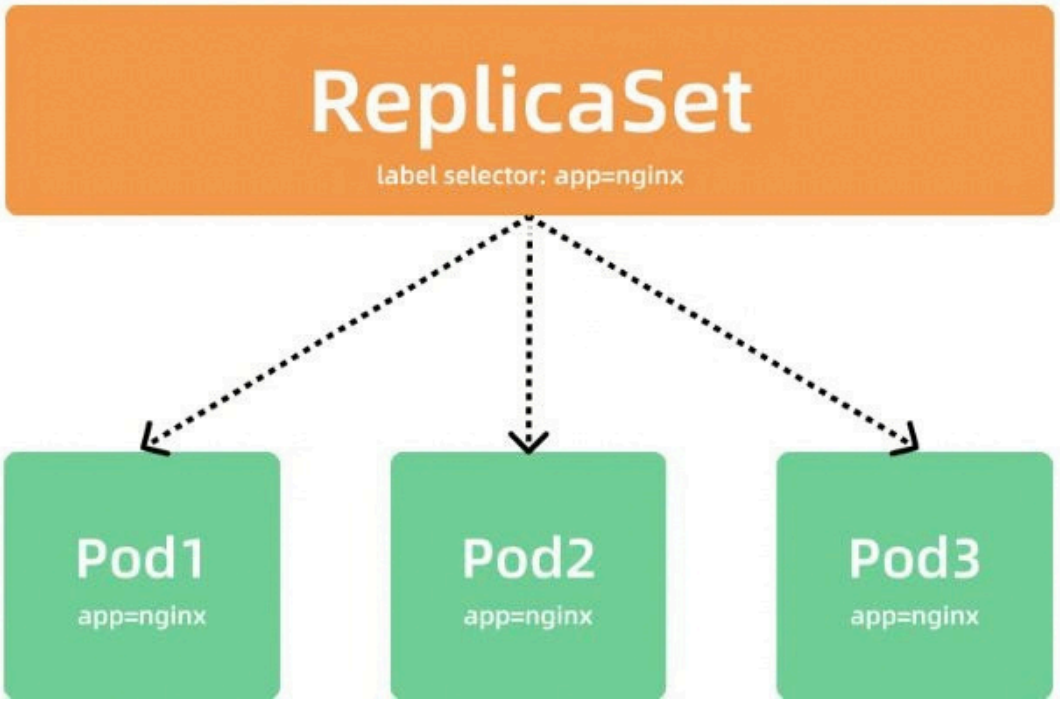
ReplicaSet

- объект k8s, который описывает в каком количестве экземпляров должен быть запущен pod – конфигурация подов описывается шаблоном
- функциональность обеспечивается соответствующим встроенным контроллером
- если количество подов больше желаемого – убивает лишние экземпляры, если меньше желаемого – запускает недостающие
- обеспечивает уникальность имен всех создаваемых подов
- для управления подами используется механизм меток и селекторов
- никак не управляет стратегиями обновления приложения

ReplicaSet. Пример манифеста

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: frontend
  labels:
    app: guestbook
    tier: frontend
spec:
  # modify replicas according to your case
  replicas: 3
  selector:
    matchLabels:
      tier: frontend
  template:
    metadata:
      labels:
        tier: frontend
    spec:
      containers:
      - name: php-redis
        image: us-docker.pkg.dev/google-samples/containers/gke/gb-frontend:v5
```

ReplicaSet. Принцип работы



Вопросы?

+ если есть вопросы

− если вопросов нет

Deployment

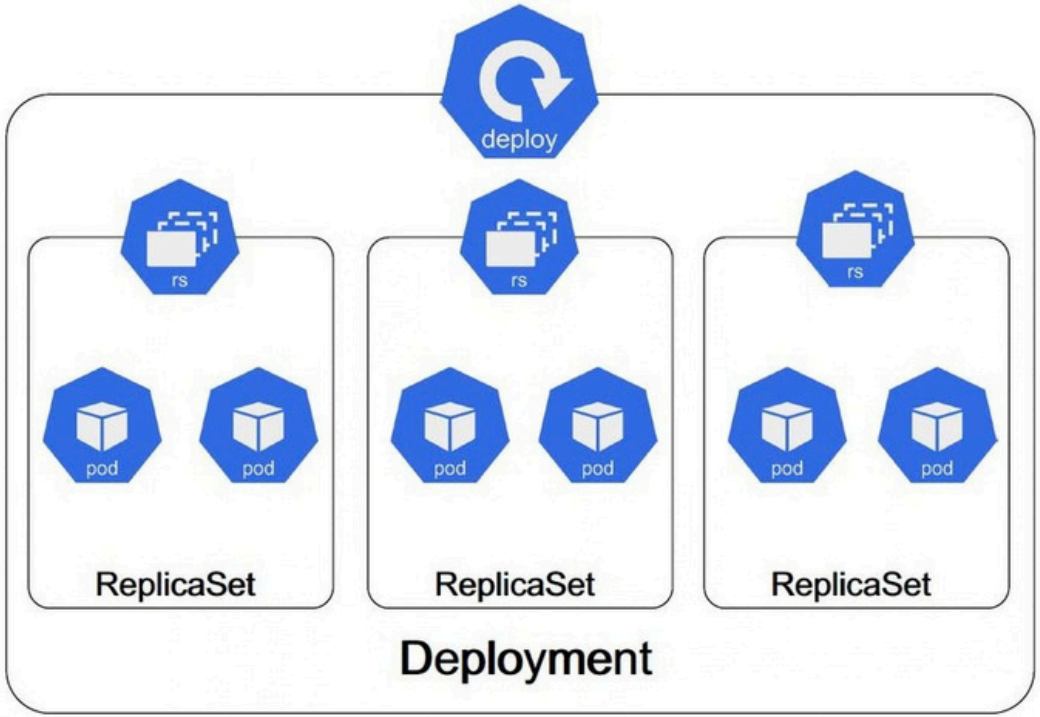
Deployment

- объект Kubernetes, который описывает в скольких экземплярах запущен сервис, а также стратегию обновления на новую версию.
- следит за этим встроенный контроллер, который позволяет не только следить за количеством реплик, но также позволяет обновлять версию и в случае ошибок откатить или остановить раскладку
- имеет две встроенные стратегии обновления – Recreate и RollingUpdate
- фактически переиспользует объект replicaSet для управления ЖЦ подов
- также как и RS для своей работы использует механизм меток и селекторов

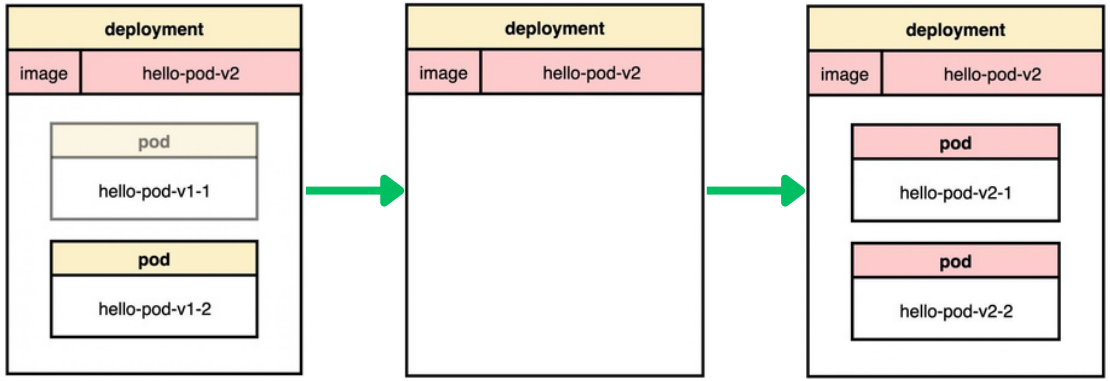
Deployment. Пример манифеста

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.14.2
        ports:
        - containerPort: 80
```

Deployment. Принцип работы

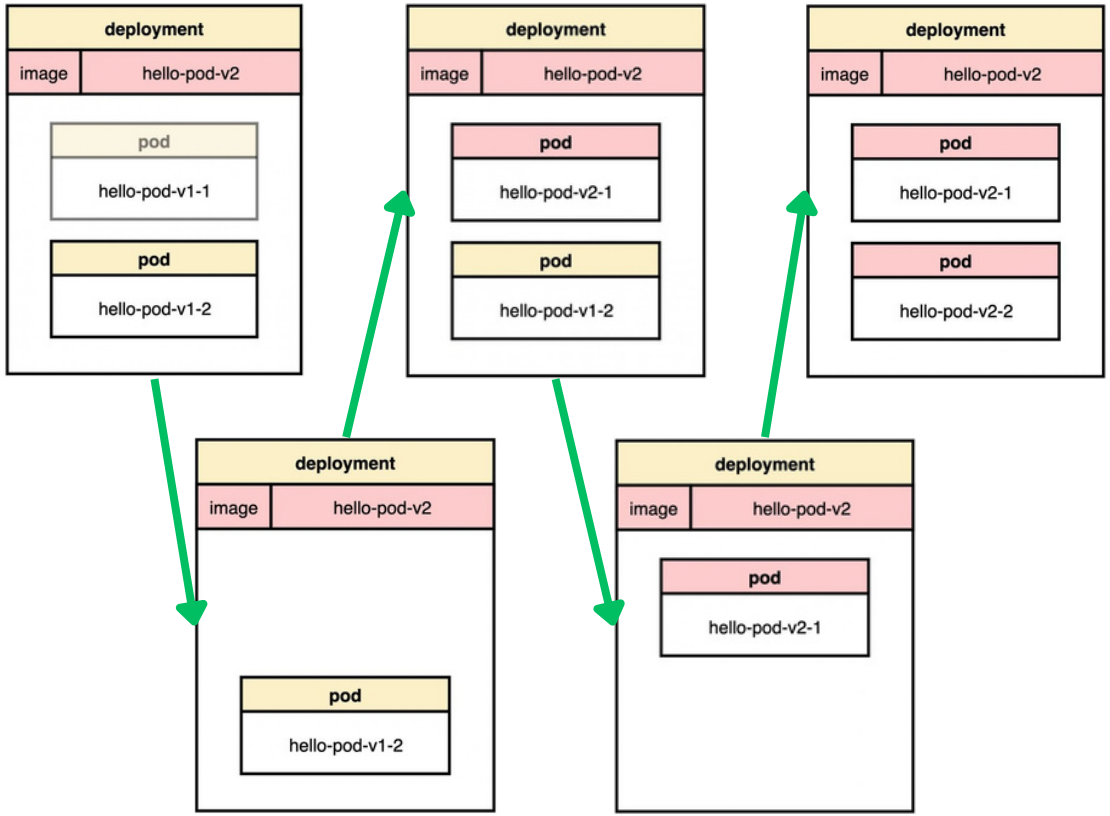


Recreate



Recreate — самый простой способ обновления. Сначала k8s завершает все Pod старой версии, затем создаёт нужное количество Pod с новой версией.

Rolling Update



RollingUpdate выполняет постепенное обновление.

Kubernetes поэтапно уменьшает количество Pod старой версии и увеличивает количество Pod новой версии.

Вопросы?

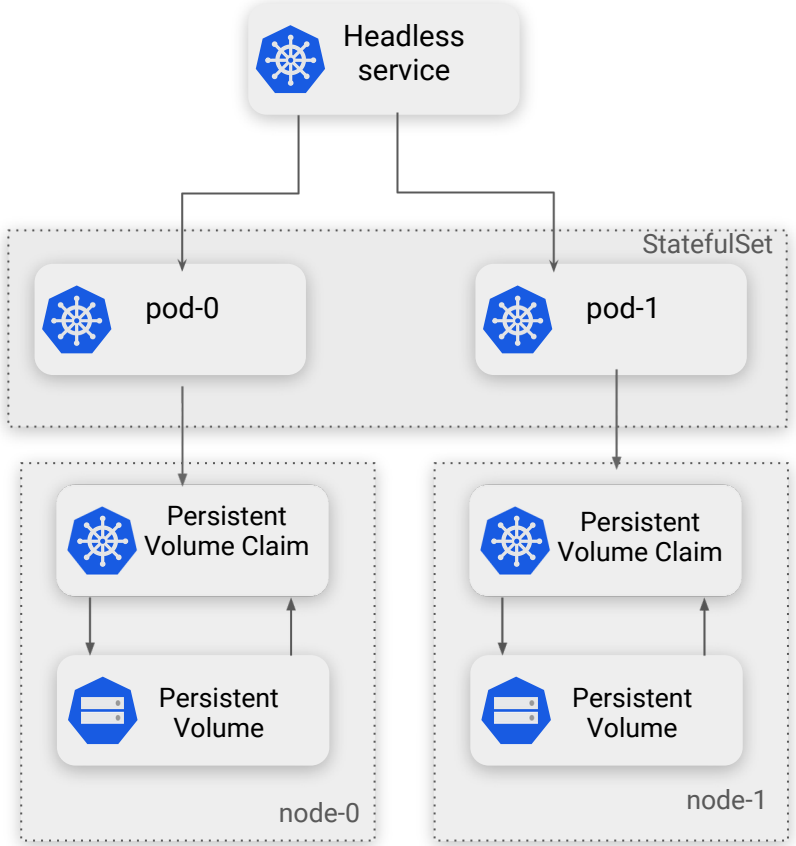
+ если есть вопросы

− если вопросов нет

StatefulSet

StatefulSet. Зачем

- Гарантии именования подов и очередности их запуска
- Отдельное персистентное хранилище для каждого пода



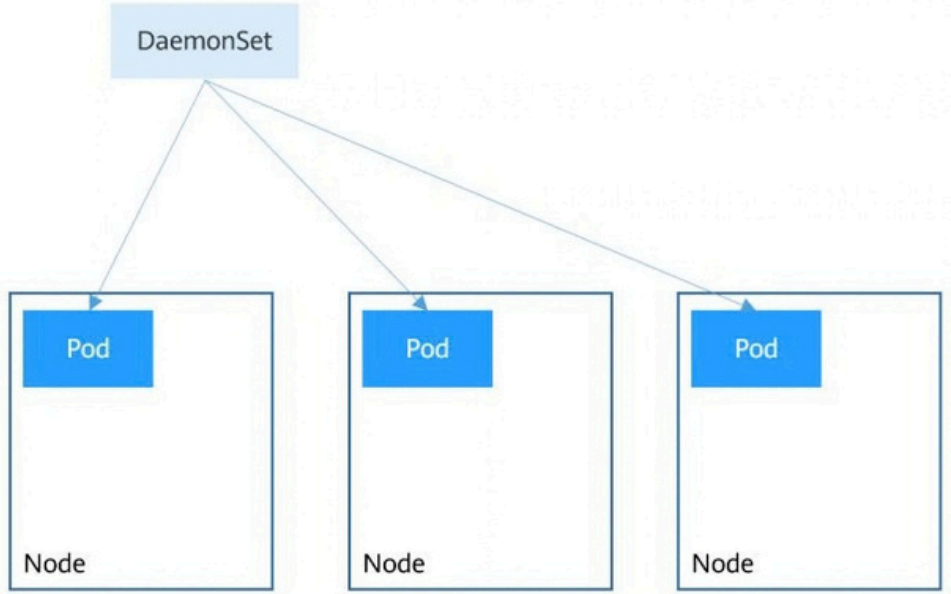
StatefulSet. Пример

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
spec:
  selector:
    matchLabels:
      app: nginx # has to match .spec.template.metadata.labels
  serviceName: "nginx"
  replicas: 3 # by default is 1
  minReadySeconds: 10 # by default is 0
  template:
    metadata:
      labels:
        app: nginx # has to match .spec.selector.matchLabels
    spec:
      terminationGracePeriodSeconds: 10
      containers:
      - name: nginx
        image: registry.k8s.io/nginx-slim:0.8
        ports:
        - containerPort: 80
          name: web
        volumeMounts:
        - name: www
          mountPath: /usr/share/nginx/html
  volumeClaimTemplates:
  - metadata:
      name: www
    spec:
      accessModes: [ "ReadWriteOnce" ]
      storageClassName: "my-storage-class"
      resources:
        requests:
          storage: 1Gi
```

DaemonSet

DaemonSet. Зачем

- Запуск агентов/демонов
- Гарантия запуска экземпляра пода на каждой ноде



DaemonSet. Пример

```yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluentd-elasticsearch
  namespace: kube-system
  labels:
    k8s-app: fluentd-logging
spec:
  selector:
    matchLabels:
      name: fluentd-elasticsearch
  template:
    metadata:
      labels:
        name: fluentd-elasticsearch
    spec:
      tolerations:
      # these tolerations are to have the daemonset runnable on control plane nodes
      # remove them if your control plane nodes should not run pods
      - key: node-role.kubernetes.io/control-plane
        operator: Exists
        effect: NoSchedule
      - key: node-role.kubernetes.io/master
        operator: Exists
        effect: NoSchedule
      containers:
      - name: fluentd-elasticsearch
        image: quay.io/fluentd_elasticsearch/fluentd:v2.5.2
```

Рефлексия

Вопросы для повторения

1. Какой контроллер следует выбрать для stateless веб-приложения с несколькими репликами и возможностью обновления без простоя?
2. Какой контроллер выбрать для PostgreSQL-кластера, где каждому экземпляру нужны стабильное имя и отдельное хранилище?
3. Какой контроллер нужен для агента сбора логов на каждой ноде?
4. Почему ReplicaSet важен для понимания Kubernetes, хотя его редко создают напрямую?
5. Как связаны между собой Deployment, ReplicaSet и Pod?
6. Какой основной принцип Kubernetes иллюстрируют все рассмотренные контроллеры?
7. Какие параметры и механизмы стоит проверить перед обновлением production-приложения через Deployment?
8. Какие риски нужно оценить перед использованием RollingUpdate?

Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу ✏️❤️